

GATE & PSU

Time Complexity Selection Series

SOLVED EXAMPLES & CRITICAL QUESTIONS

| Q1: RECURRENCE SUB-PROBLEM

The Equation

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{2n}{5}\right) + 7n$$

Given $T(0) = 1$. Determine asymptotic complexity.

The Solution

Total sub-problem size per level decreases geometrically:

$$\frac{n}{2} + \frac{2n}{5} = \frac{9n}{10}$$

Geometric Series total work is dominated by the root level.

Final: $\Theta(n)$

Q2: MASTER THEOREM LOGIC



Goal

Hold $T(n) = \Theta(n \log n)$



Case 2

$n^{\log_b a} = f(n)$



Condition

$a = b$ AND $f(n) = \Theta(n)$

| Q3: SINGLE-PASS CHECK

Algorithm check: Is an array of size N sorted? (Single pass, adjacent comparison).

Logic Breakdown

- Comparisons: $N - 1$ pairs
- Each comparison: $O(1)$
- Total: $\Theta(N)$

$\Theta(N)$

Tightest Asymptotic Bound

| Q4: SERIES EVALUATION

$$1^3 + 2^3 + \dots + n^3 = \frac{n(n+1)^2}{2}$$

Valid Bounds

- ✓ $\Theta(n^4)$ - Exact
- ✓ $O(n^5)$ - Upper
- ✓ $\Omega(n^3)$ - Lower

Invalid Bounds

- ✗ $\Theta(n^5)$ - False order

Q5: LOGARITHMIC DEPENDENCIES

```
for(i=1; i<=n; i=i*2)
    ++p;
    for(k=1; k<=i; k=k*2)
```

Execution Analysis

1. $p = \Theta(\log n)$
2. Inner Loop (k) = $\log p = \log \log n$
3. Outer Loop (i) = n times

Result: $\Theta(n \log \log n)$

Q6: HALVING INNER RANGE

```
for(int i=n; i>0; i/=2) {  
    for(int j=0; j<=i; j++)  
        sum++;  
}
```

Mathematical View

Work follows Geometric Progression:

$$n + \frac{n}{2} + \frac{n}{4} + \dots$$

The sum converges to $2n$.

$$\Theta(n)$$

Q7: ARITHMETIC PROGRESSION

```
for(int i=1; i≤n; i++) {  
    for(int j=1; j≤i; j++) {  
        // constant work  
    }  
}
```

Summation

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Dominant term is n^2 .

$$\Theta(n^2)$$

Q8: BRANCHING RECURRENCE

```
void fun(int n) {  
    if(n ≤ 1) return;  
    fun(n/2);  
    fun(n/2);  
}
```

Recurrence Formulation

$$T(n) = 2T(n/2) + 1$$

Using Master Theorem (Case 1):

$$\Theta(n)$$

Note: Work done at the leaves.

Q9: RECURSIVE FIBONACCI

```
int fib(int n) {  
    if(n ≤ 1) return n;  
    return fib(n-1) + fib(n-2);  
}
```

✓ Recurrence: $T(n) = T(n-1) + T(n-2) + 1$

📊 Solution: Golden Ratio Base (Φ)

⚡ Final Answer: $\Theta(1.618^n)$

| Q10: INCREASING ORDER

Rank	Function	Category
1 (Slowest Growth)	$n \log n$	Linearithmic
2	$n^{3/2}$	Polynomial
3	$n^{\log_2 n}$	Quasi-polynomial
4 (Fastest Growth)	2^n	Exponential

| Q11: SIMPLE LINEAR

Recurrence

$$T(n) = T(n-1) + 1$$

Expanding gives:

$$1 + 1 + \dots + 1 \text{ (n times)}$$

$$\Theta(n)$$

Q12: RECURSIVE SUM

Recurrence

$$T(n) = T(n-1) + n$$

Expansion:

$$n + (n-1) + \dots + 1$$

$$\Theta(n^2)$$

| Q13: EXPANSION GROWTH

$$T(n) = 2T(n-1) + 1$$

Expansion gives:

$$2^n T(0) + (2^n - 1)$$

$$\Theta(2^n)$$

| Q14: SINGLE HALVING

$$T(n) = T(n/2) + 1$$

Number of levels in a binary search style tree:

$$\Theta(\log n)$$

Q15: LINEARITHMIC LOOP

```
for(int i=1; i≤n; i*=2) {  
    for(int j=1; j≤n; j++) {  
        // constant  
    }  
}
```

Analysis

</> Outer: $\log n$ steps

</> Inner: n steps

 Total: $\log n * n$

$\Theta(n \log n)$